Инструмент автоматизации проектирования

архитектуры микросервисных систем

AutoSystemDesign

Дичковский Владимир

github.com/Volodya1000/AutoSystemDesign

Возможности инструмента

Построение архитектуры

- Формирование архитектурной модели по функциональным и нефункциональным требованиям.
- Генерация C4-диаграммы контейнеров и диаграмм последовательностей.
- Формирование спецификации API.

Проверка и анализ

- Проверка структуры и связей архитектурной модели.
- Расчётная оценка нагрузки и задержек.
- Извлечение требований из документа с цитатами, подтверждающими исходный текст.

Результаты формируются в виде архитектурных моделей и связанных с ними артефактов.

Входные данные и результаты

Пользователь передаёт на вход:

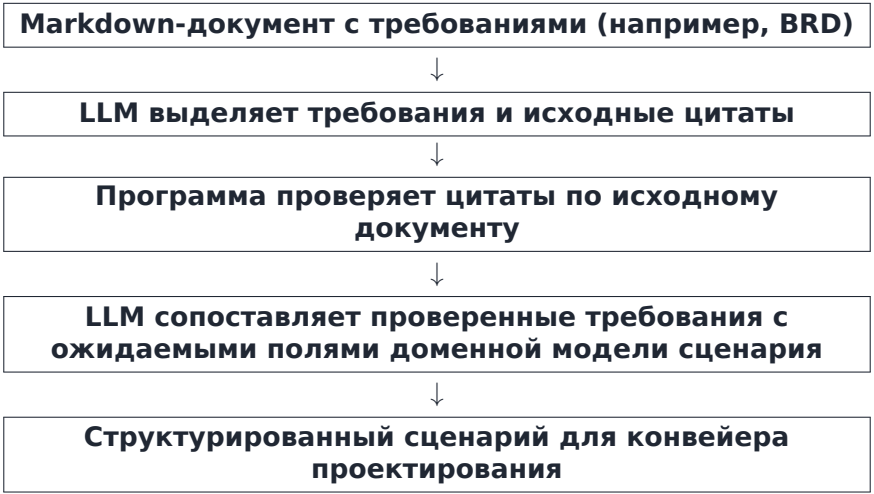
- Документ с требованиями (например, BRD в Markdown)
- Функциональные требования
- Нефункциональные требования
- Профиль нагрузки

Пользователь получает на выход:

- C4 Container Diagram
- Sequence-диаграмма на каждое функциональное требование
- OpenAPI 3.1 и Swagger UI

Вход можно задать в YAML или передать Markdown-документ с требованиями. Поддержка PDF, DOCX и других форматов пока не реализована.

Извлечение требований из документа



Проверка достоверности извлечённых данных

Правила проверки

- Каждая цитата должна присутствовать в исходном тексте.
- Значения полей сценария сопровождаются проверенными цитатами.
- Числа должны буквально встречаться в цитате: пересчёт единиц не выполняется.

Если подтверждения нет

- Неподтверждённые утверждения и значения исключаются с указанием причины.
- Неоднозначности и противоречия сохраняются для проверки человеком.

Для извлечённых значений сохраняются исходные цитаты и результаты проверки.

Выходные артефакты

C4-диаграмма контейнеров

- Скомпилированное изображение в выбранном формате (PNG/SVG)
- Исходный код PlantUML
- JSON-модель архитектуры

Диаграмма последовательностей

- Скомпилированное изображение в выбранном формате (PNG/SVG)
- Исходный код PlantUML
- JSON доменной модели диаграммы последовательно-стей

Спецификация API

- Спецификация OpenAPI в формате JSON
- Swagger UI в формате HTML

Изображения предназначены для просмотра, PlantUML — для редактирования диаграмм, JSON — для повторного использования структурированных моделей.

Варианты сервера рендеринга PlantUML

Для компиляции исходного кода PlantUML в изображение доступны три варианта:

1. **Локальный сервер PlantUML, развёртываемый через Docker Compose**
   Развёртывается вместе с проектом; конфигурация включена в репозиторий.
2. **Официальный сервер PlantUML**
   Используется размещённый разработчиками PlantUML сервер.
3. **Kroki**
   Универсальный сервис рендеринга диаграмм, поддерживающий PlantUML и другие форматы.

Генерация и проверка



Ключевые принципы

- LiteLLM позволяет обращаться к разным моделям через OpenAI-совместимый API.
- LLM формирует структурированный вывод по классам доменной модели; код PlantUML она не пишет.
- Доменная модель описывает архитектуру, функциональные требования и спецификацию API.
- Сейчас диаграммы строятся через PlantUML. Структурированная модель позволяет добавить другой рендерер, например Mermaid, отдельно от генерации модели.

Повторная генерация при ошибках

1. LLM формирует структурированный ответ по классам доменной модели.
2. Ответ разбирается как JSON и преобразуется в типизированные модели.
3. При ошибке JSON-разбора в контекст добавляется описание ошибки.
4. Если модель не проходит доменные валидаторы, в контекст добавляется форматированное описание нарушенных правил.
5. LLM получает следующую попытку с учётом ошибки и исправляет ответ.

Число попыток ограничено настройкой. Если лимит исчерпан, генерация завершается ошибкой, а не запускается бесконечно.

Модель C4 на уровне контейнеров

Типы узлов

- Клиент
- Балансировщик нагрузки
- Вычислительный узел
- База данных
- Кэш
- Брокер сообщений

Параметры, задаваемые LLM

- Число реплик, параллелизм и размер очереди
- Среднее и стандартное отклонение времени обработки
- Оценка месячной стоимости узла

Использование параметров

- Значения проверяются по типам и ограничениям доменной модели.
- Используются в расчётах пропускной способности, задержек и бюджета.

Стоимость задаётся в модели, а не получается из актуального каталога облачного провайдера.

Онтология связей между узлами

Пример разрешённых связей

- Клиент → вычислительный узел
- Вычислительный узел → база данных

Пример недопустимых связей

- Клиент → база данных
- База данных → вычислительный узел

Онтология задаёт допустимые пары типов узлов; проверка выполняется до построения диаграммы.

Протоколы взаимодействия между узлами

- LLM выбирает протокол для каждой связи: HTTP (REST), gRPC, TCP или Message Queue.
- Протокол и признак асинхронности сохраняются в модели и отражаются на C4-диаграмме.
- При расчёте критического пути учитываются только синхронные связи; сетевая задержка берётся из отдельного параметра связи.
- Для диаграмм последовательностей валидатор проверяет, что события и уведомления отправляются асинхронно.

Планируется добавить: Webhooks, Long Polling, Server-Sent Events (SSE).

Реализованные валидаторы C4-модели

Структура графа

- У каждого узла уникальный идентификатор.
- Каждая связь соответствует разрешённой паре типов узлов.
- Граф не содержит циклов.
- Каждый узел связан хотя бы с одним другим узлом.

Реализованные валидаторы C4-модели

Валидация семантики

- Брокер сообщений должен быть связан с узлом-производителем и узлом-потребителем сообщений.
- При использовании кэша должен быть определён путь обращения к основному хранилищу при промахе кэша.

Валидация на основе расчётов

- Рассчитывается загрузка узлов с ограниченной пропускной способностью с учётом RPS, числа реплик и параллелизма.
- Средняя задержка синхронного критического пути оценивается с учётом очередей и сетевых задержек и сравнивается с заданным порогом.

Допущения и дальнейшее развитие анализа нагрузки

Текущая оценка

- Поток поровну распределяется между клиентами и исходящими связями; для кэша учитывается доля промахов.
- Задержка оценивается аналитически по параметрам узлов и сетевым задержкам.
- Проверяется средняя задержка; p95/p99 и симуляция нагрузки пока не рассчитываются.

Следующие направления

- Развитие моделей очередей и каналов обслуживания.
- Отдельная проверка SLO по перцентилям.
- Симуляция нагрузки с помощью SimPy.

Реализованные валидаторы диаграммы последовательностей

Текущие правила

- На каждое функциональное требование — отдельная диаграмма.
- Ссылки на существующие роли и узлы C4.
- Соответствие сценария исходному требованию.
- Использование всех объявленных линий жизни.
- Разные бизнес-роли не используют один клиентский компонент.
- События и уведомления должны быть асинхронными.

Дальнейшее развитие

Расширение проверок семантики сообщений и ветвлений сценария.

REST API: OpenAPI и Swagger

OpenAPI 3.1

- Операции и маршруты
- Параметры и схемы
- Связь с требованиями и узлами C4

Проверки

- Уникальность операций
- Коллизии маршрутов
- Согласованность ссылок

Интерфейс и способы запуска

Текущая реализация

- Консольный интерфейс для запуска сценариев генерации и извлечения требований из Markdown-документов; параметры генерации настраиваются.

Возможные направления развития

- Добавить FastAPI, чтобы запускать инструмент как сервер и предоставлять его функции через API.
- Разработать отдельный фронтенд для настройки сценариев и просмотра результатов.

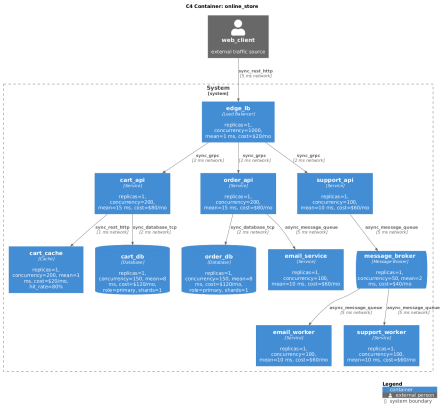
Пример входного YAML

social_network_chat.yaml

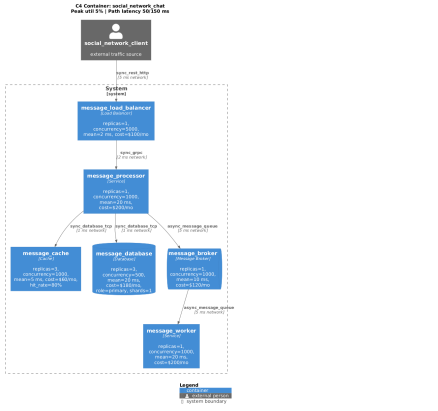
```yaml
project_name: "social_network_chat"
description: "High-load messaging module for a social network platform"
workload:
  baseline_requests_per_second: 500.0
  peak_requests_per_second: 2500.0
  read_ratio: 0.6
requirements:
  - type_identifier: latency
    threshold_value: 150.0
    percentile: 99.0
  - type_identifier: availability
    threshold_value: 99.99
    operator: ">="
functional_requirements:
  - identifier: "req_msg_01"
    name: "Direct Message Delivery"
    text: >
      Sender sends a direct message to a recipient. The system checks the recipient's presence status.
      If the recipient is online, the message is delivered immediately via an active WebSocket connection.
      If the recipient is offline, the message is queued and a Push Notification is sent to their device.
  - identifier: "req_msg_02"
    name: "Read Receipt Acknowledgment"
    text: >
      When the recipient opens the chat window, the client application sends a read receipt event.
      The system updates the message status in the database and pushes the updated status back to the sender.
```

Примеры C4-диаграмм

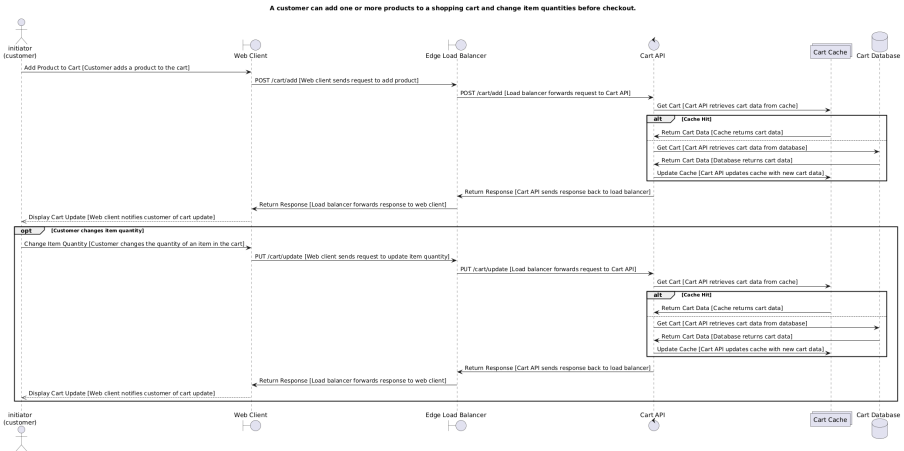
Пример C4-диаграммы 1



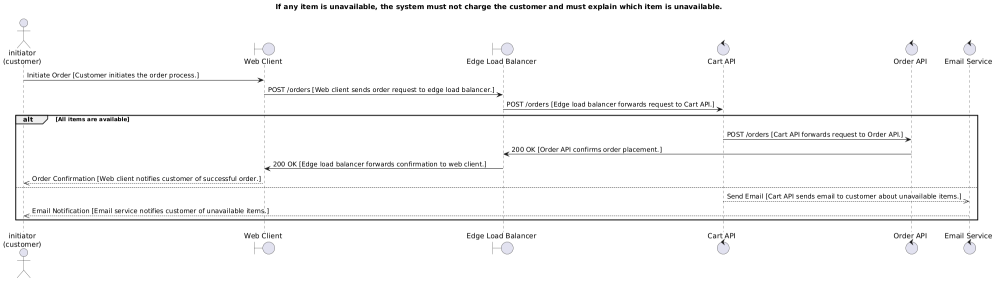
Пример C4-диаграммы 2



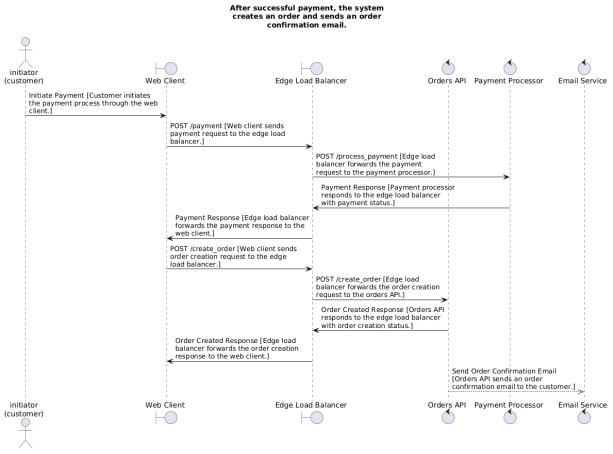
Пример диаграммы последовательностей 1



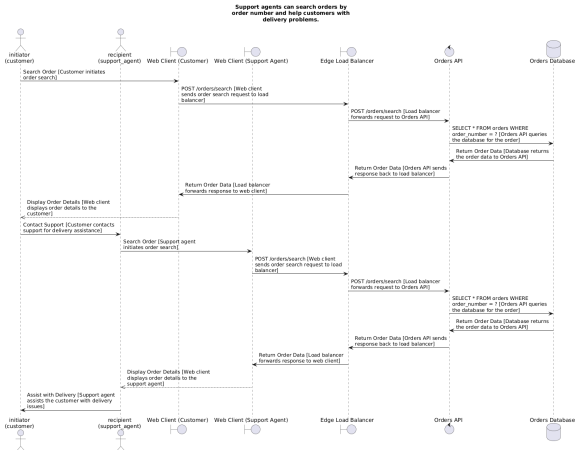
Пример диаграммы последовательностей 2



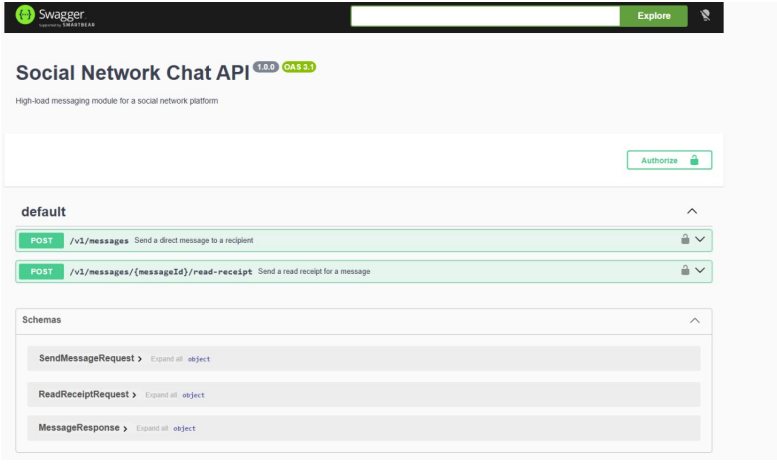
Пример диаграммы последовательностей 3



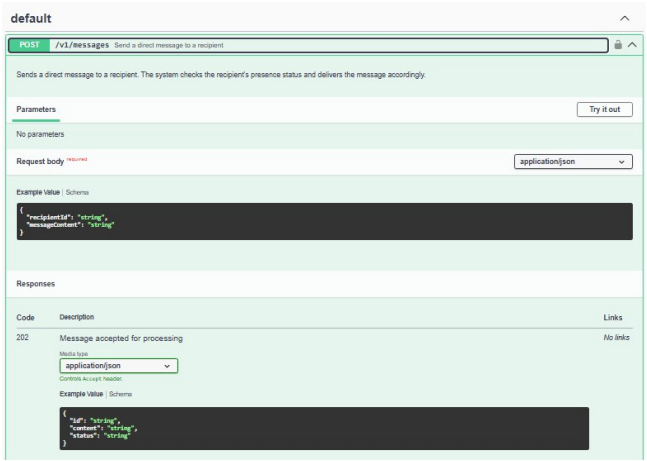
Пример диаграммы последовательностей 4



Пример спецификации Swagger



Пример спецификации Swagger 2



Пример спецификации Swagger 3

